

# LAB: CAP-A-Pizza

## KI-Werkzeuge & Coding-Agents (Erlaubt und Erwünscht!)

In diesem Praktikum geht es um Makro-Architektur, nicht um das Auswendiglernen von Framework-Syntax. Der Einsatz von LLMs (ChatGPT, Claude) oder Coding-Agents ist ausdrücklich erlaubt! Ihre Rolle: Sie sind der Architekt, die KI ist Ihr Junior-Entwickler.

- Die KI kann Ihnen den RabbitMQ-Boilerplate-Code generieren.
- **Aber:** Sie müssen der KI genau sagen, welche Events geworfen werden, Sie müssen die Saga-State-Machine vorher aufzeichnen und Sie müssen am Ende anhand der Logs beweisen können, dass der Architektur-Fluss (z.B. die kompensierende Transaktion) korrekt funktioniert. **Wer den Code seiner KI nicht erklären kann, hat das Architektur-Ziel verfehlt.**

## Quickstart

```
$git clone https://github.com/wuerzig/cap-a-pizza.git  
$cd cap-a-pizza  
$less INSTALL.md
```

## Aufgabe 1: Conway & CAP

### Conway's Gesetz (20 Minuten)

- **Gruppenbildung:** Finden Sie sich in 3er-Gruppen zusammen. Jedes Teammitglied übernimmt exakt eine Service-Domäne: OrderService, KitchenService oder PaymentService.
- **Die Regel:** In den ersten 20 Minuten herrscht **absolutes Sprechverbot**.
- **Die Aufgabe:** Verhandeln Sie die API-Verträge (JSON-Payloads für das Bestellen, Bezahlen und Backen einer Call-a-Saga-Pizza) ausschließlich durch das Weiterreichen von Post-its.
- **Auswertung:** Brechen Sie das Schweigen und diskutieren Sie, wie diese Kommunikationsbarriere das API-Design und die Kopplung Ihrer Services beeinflusst hat.

### **CAP-Theorem (50 Minuten)**

- **Setup:** Starten Sie die bereitgestellte Infrastruktur mit `docker-compose up -d`. Dies startet ein MongoDB Replica Set (3 Knoten), Message Broker und das Dashboard.
- **Hohe Verfügbarkeit**
  1. Prüfen Sie, dass in der `.env`-Datei des OrderService die Konfiguration `MONGO_WRITE_CONCERN=1` gesetzt ist.
  2. Starten Sie das Skript `load_tester.py` und beobachten Sie die eingehenden Bestellungen.
  3. Simulieren Sie eine Netzwerkpartition: Öffnen Sie ein neues Terminal und pausieren Sie einen sekundären Datenbankknoten mit `docker pause mongo-sec-1` (`mongo-sec-2`)
  4. Beobachten Sie: Das System nimmt weiterhin Bestellungen an (Verfügbarkeit bleibt erhalten, absolute Konsistenz im Cluster wird geopfert).
- **Höhere Konsistenz**
  1. Heben Sie die Pausierung auf (`docker unpause mongo-sec-1 mongo-sec-2`), ändern Sie die Konfiguration auf `MONGO_WRITE_CONCERN=3` und starten Sie den Service neu (`docker compose restart order-service`).
  2. Starten Sie den Load-Tester erneut.
  3. Pausieren Sie wieder `mongo-sec-1` oder `mongo-sec-2`.
  4. Beobachten Sie: Das System akzeptiert Aufträge, markiert diese jedoch als "NOT REPLICATED"
- **Strikter Fehler**
  1. Pausieren Sie beide Knoten `mongo-sec-1` UND `mongo-sec-2` - Was passiert, und warum?

## **Aufgabe 2: EDA & CQRS**

### **Von REST zum Event-Bus**

Aktuell wartet der OrderService bei jeder Bestellung synchron auf den KitchenService. Wenn die Küche überlastet ist, staut sich das gesamte System und Requests laufen in einen Timeout.

### **Teil 1**

Suchen Sie im OrderService nach dem Kommentar `// TODO` (Lab 2). Entfernen Sie den

synchronen HTTP-Call. Implementieren Sie stattdessen einen Publisher, der ein OrderPlacedEvent an den bereitgestellten RabbitMQ-Broker sendet. Schreiben Sie dann im KitchenService einen Listener, der dieses Event konsumiert und den Pizza-Backvorgang asynchron startet.

## Teil 2: Das CQRS Dashboard (45 min)

Das Management-Dashboard von Call-a-Saga blockiert die produktive Datenbank durch ständige, komplexe Lesezugriffe (GET /orders). Trennen Sie Lese- und Schreibvorgänge (Command Query Responsibility Segregation)!

- Schreiben Sie ein neues Modul, das auf alle relevanten Events (OrderPlaced, PizzaBaked) hört und daraus kontinuierlich ein hochoptimiertes Lese-Modell (Read Model) in einem In-Memory-Dictionary oder Redis-Cache aktualisiert.
- Bauen Sie den GET /orders Endpunkt so um, dass er ausschließlich aus diesem schnellen Cache liest.

## Aufgabe 3: Sagas (90 min)

### Design (OHNE LLM/AGENT/KI!)

Neue Business-Regel: Pizzen dürfen erst gebacken werden, wenn die Bezahlung erfolgreich war. Der PaymentService simuliert nun realistisches Verhalten und lehnt zufällig 30% aller Zahlungen ab (z.B. mangelnde Deckung, Fehler beim Provider). Da die Pizzeria nun Event-gesteuert arbeitet, können Sie keine einfachen SQL-Rollbacks über Service-Grenzen hinweg nutzen.

**Aufgabe:** Zeichnen Sie als Team den Zustandsautomaten (State Machine) für eine **Orchestration Saga** (auf Papier). Beziehen Sie Zustände wie Pending -> Awaiting Payment -> (Paid & Baking) ODER (Payment Failed & Cancelled) ein.

Wenn Sie KI/LLMs verwenden, erzeugen Sie aus Ihrem Entwurf ein Prompt.

### Implementation

Implementieren Sie den gezeichneten Saga-Orchestrator im Code.

- Lassen Sie den Orchestrator das PaymentFailedEvent abonnieren.
- Wenn dieses Event eintrifft, muss eine **kompensierende Transaktion** ausgeführt werden: Der Orchestrator sendet ein CancelOrderCommand an den OrderService.
- Stellen Sie sicher, dass im OrderService die Bestellung storniert wird, geblockte Ressourcen freigegeben werden und Ihr CQRS-Read-Modell aus Praktikum 2 den Status sofort auf "Declined/Abgelehnt" aktualisiert.